

Série 1 : Programmation orienté Objet

Exercice 1 :

Écrire en Python une classe «Rectangle» ayant deux variables « a » et « b » et une fonction membre « surface() » qui retournera la surface du rectangle.

Exercice 2 :

Écrire en Python une classe « Somme » ayant deux variables « n1 » et « n2 » et une fonction membre « som() » qui calcule la somme. Dans la méthode principale main demandez à l'utilisateur d'entrez deux entiers et passez-les au constructeur par défaut de la classe « Somme » et afficher le résultat de l'addition des deux nombres.

Exercice 3 :

Écrire classe Python appelée « Etudiant » avec les membres suivant :

nom : (de type char),

note1, note2 : (de type float)

calc_moy() : calcule la note moyenne.

afficher () : affiche le nom et la note moyenne.

Le programme principal (main) demande à l'utilisateur d'entrer le nom et les notes d'un étudiant. et affiche leur nom et la note moyenne.

Exercice 4 :

Réaliser en Python une classe point permettant de manipuler un point d'un plan. on prévoira :

1) un point est défini par ses coordonnées x et y (des membres privés)

2) les constructeurs

3) une fonction membre déplace effectuant une translation définie par ses deux arguments dx et dy (double)

4) une fonction membre affiche se contentant d'afficher les coordonnées cartésiennes du point.

5) une fonction membre saisir se contentant de saisir les coordonnées cartésiennes du point.

6) une fonction membre distance effectuant calculant la distance entre deux point.

7) une fonction membre milieu donnant le milieu d'un segment.

8) un petit programme d'essai (main) gérant la classe point.

Exercice 5 :

Écrire un programme en Python qui simule la gestion d'un simple compte bancaire. Le compte est créé avec un solde initial. Il est possible de déposer et de retirer des fonds, d'ajouter des intérêts et de connaître le solde actuel. Cela devrait être implémenté dans une classe nommée Account qui comprend:

1) Un constructeur par défaut qui met le solde initial à zéro.

2) Un constructeur qui accepte une balance initial comme paramètre.

- 3) Une fonction `getBalance` qui renvoie le solde actuel.
- 4) Une méthode `deposer` pour déposer un montant spécifié.
- 5) Une méthode `retirer` pour retirer un montant spécifié.
- 6) Une méthode `ajouter_Interet` pour ajouter de l'intérêt au compte.

La méthode `ajouter_Interet` prend le taux d'intérêt comme paramètre et modifie le solde du compte en $\text{solde} * (1 + \text{taux d'intérêt})$.

Exercice 6 :

Créer en Python une classe appelée `Temps`, qui a des membres de type `int` tels que heures, minutes et secondes. (rendez-les `private`)

- 1) Un constructeur doit initialiser ces données à 0
- 2) Un autre constructeur devrait l'initialiser à des valeurs fixes.
- 3) Une fonction membre devrait l'afficher, au format 17h 59min 59s.
- 4) Une autre fonction pour renvoyer les données de chaque membre nommez-les `getheurs`, `getMin` et `getSec`
- 5) Une fonction membre doit ajouter deux objets de type `Temps` passé en arguments.

Exercice 7:

Écrire en Python un programme utilisant une classe `rectangle` dont le constructeur prend deux paramètres, largeur et hauteur et qui offre les fonctions suivantes :

- 1) calcul du périmètre
- 2) calcul de la surface
- 3) affichage

ainsi que les accesseurs et mutateurs triviaux (lecture et modification de la largeur et de la hauteur).

Exercice 8:

Écrivez un programme en Python qui définit une classe appelée `Forme` avec un constructeur qui donne de la valeur à la largeur(`x`) et à la hauteur(`y`). Définir la méthode `aire()` dans les deux sous-classes `Triangle` et `Rectangle`, qui calculent l'aire. Dans la méthode principale `main`, définissez deux variables, un triangle et un rectangle, puis appelez la fonction `aire()` dans ces deux variables.

Notez que:

l'aire du triangle est = $\text{largeur} * \text{hauteur} / 2$

l'aire du rectangle est = $\text{largeur} * \text{hauteur}$.

Exercice 9:

- 1) Ecrire une classe `Rectangle` en langage Python, permettant de construire un rectangle dotée d'attributs longueur et largeur.
- 2) Créer une méthode `Perimetre()` permettant de calculer le périmètre du rectangle et une méthode `Surface()` permettant de calculer la surface du rectangle
- 3) Créer les getters et setters.
- 4) Créer une classe fille `Parallelepipede` héritant de la classe `Rectangle` et dotée en plus d'un attribut hauteur et d'une autre méthode `Volume()` permettant de calculer le volume du Parallélépipède.

Exercice 10 :

- 1) Créer une classe Python nommée `CompteBancaire` qui représente un compte bancaire, ayant pour attributs : `numeroCompte` (type numérique) , `nom` (nom du propriétaire du compte du type chaîne), `solde`.
- 2) Créer un constructeur ayant comme paramètres : `numeroCompte`, `nom`, `solde`.
- 3) Créer une méthode `Versement()` qui gère les versements.
- 4) Créer une méthode `Retrait()` qui gère les retraits.
- 5) Créer une méthode `Agios()` permettant d'appliquer les agios à un pourcentage de 5 % du solde
- 6) Créer une méthode `afficher()` permettant d'afficher les détails sur le compte
- 7) Donner le code complet de la classe `CompteBancaire`.

Exercice 11:

- 1) Définir une classe `Cercle` permettant de créer un cercle `C(O,r)` de centre `O(a,b)` et de rayon `r` à l'aide du constructeur :
- 2) Définir une méthode `Surface()` de la classe qui permet de calculer la surface du cercle
- 3) Définir une méthode `Perimetre()` de la classe qui permet de calculer le périmètre du cercle
- 4) Définir une méthode `testAppartenance()` de la classe qui permet de tester si un point `A(x,y)` appartient ou non au cercle `C(O,r)`.

Exercice 12 :

- 1) Créer une classe `Calcul` ayant un constructeur par défaut (sans paramètres) permettant d'effectuer différents calculs sur les nombres entiers.
- 2) Créer au sein de la classe `Calcul` une méthode nommée `Factorielle()` qui permet de calculer la factorielle d'un entier. Tester la méthode en faisant une instantiation sur la classe.
- 3) Créer au sein de la classe `Calcul` une méthode nommée `Somme()` permettant de calculer la somme des `n` premiers entiers: $1 + 2 + 3 + \dots + n$. Tester la méthode.
- 4) Créer au sein de la classe `Calcul` une méthode nommée `testPrim()` permettant de tester la primalité d'un entier donné. Tester la méthode.
- 5) Créer au sein de la classe `Calcul` une méthode nommée `testPrims()` permettant de tester si deux nombres sont premiers entre eux.
- 6) Créer une méthode `tableMult()` qui crée et affiche la table de multiplication d'un entier donné. Créer ensuite une méthode `allTablesMult()` permettant d'afficher toutes les tables de multiplications des entiers 1, 2, 3, ..., 9.
- 7) Créer une méthode `listDiv()` qui récupère tous les diviseurs d'un entier donné sur une liste `Ldiv`. Créer une autre méthode `listDivPrim()` qui récupère tous les diviseurs premiers d'un entier donné.

Exercice 13

Coder une classe `myString` permettant de doter les chaînes de caractères des méthodes `append()` et `pop()` faisant les mêmes opérations que celles des listes. Exemple si on crée des chaînes via l'instanciation `s1 = myString("Hello")` et `s2 = "bonjour"`, et on lui applique les méthodes :

```
print(s1.append(" world !")) # affiche 'Hello world !'
print(s2.pop(2)) # affiche 'bojour'.
```

Exercice 14

1. Définir une classe Book avec les attributs suivants : Titre, Auteur (Nom complet), Prix.
2. Définir un constructeur ayant comme attributs: Titre, Auteur, Prix.
3. Définir la méthode View() pour afficher les informations d'une instance object Book.
4. Ecrire un programme pour tester la classe Book.

Exercice 15:

Ecrire une classe Python nommée Geometry avec un constructeur par défaut sans paramètres.

- 1) Ajouter une méthode nommée distance() à la classe geometry qui permet de calculer la distance entre deux points

$A = (a1, a2)$, $B = (b1, b2)$ (avec la convention: un point est identifié à ses coordonnées $M = (xM, yM)$)

- 2) Ajouter une méthode nommée middle() à la classe geometry qui permet de déterminer le milieu d'un bipoint (A , B).

- 3) Ajouter une méthode nommée trianglePerimeter() à la classe geometry qui permet de calculer le périmètre d'un triangle ABC.

- 4) Ajouter une méthode nommée triangleIsoscel() qui renvoie True si le triangle est isoscel et False sinon.